
WLCF

Release 1.0.0

Alejandro Avilés

May 06, 2026

CONTENTS:

1	Overview	1
1.1	wlcf: Weak Lensing Correlation Function	1
1.2	Introduction	1
1.3	Compiling and getting started	1
1.4	License	2
1.5	Acknowledgements	2
2	WLCF Installation	3
2.1	Requirements	3
2.2	Compilation	3
3	Parameters	7
3.1	Parameters related to the cosmological model	7
3.2	Parameters to control the input power spectrum	7
3.3	Parameters related to the weak lensing kernel	7
3.4	Parameters controlling the model and numerical setup	8
3.5	Parameters to control output files	8
3.6	Miscellaneous parameters	9
4	Linear power spectrum files	11
5	Pre/Post processing	13
5.1	Pre-processing	13
5.2	Post-processing	13
6	AddOn's	15
7	Python interface	17
8	Hands on	19
8.1	Setting up the working directory	19
8.2	Generating the linear power spectrum	19
8.3	Running wlcf from Python	21
8.4	Plotting the 3PCF multipoles	22
8.5	Comparing different models	25
9	3-point correlation functions	29
10	Indices and tables	31

OVERVIEW

1.1 wlcfc: Weak Lensing Correlation Function

Authors: A. Aviles, J.C. Hidalgo, E.A. Moreno-Alcala, G. Niz, S. Ramirez, Mario A. Rodriguez-Meza, S. Samario-Nava

For download and information, see <https://github.com/rodriguezmeza/wlcfc.git> .

1.2 Introduction

wlcfc is a C code designed to compute cosmological correlation functions using theoretical frameworks such as Standard Perturbation Theory (SPT), Effective Field Theory (EFT), and the Halo Model. Currently, it focuses on the computation of the **three-point correlation function (3PCF)** for galaxy weak lensing convergence in a plane-wave expansion. Paper available at (<https://arxiv.org/abs/2408.16847>).

1.3 Compiling and getting started

Download the code by cloning it from: <https://github.com/rodriguezmeza/wlcfc.git> .

Dependencies: *wlcfc* optionally requires **GSL (GNU Scientific Library)** and **FFTW3** installed in your system. Make sure to adapt the paths to these libraries in *Makefile_machine*.

To run the code go to the *wlcfc* directory and compile the code:

```
$ cd wlcfc
$ make clean
$ make all
```

The code has been tested with modern versions of *gcc*. OpenMP support is optional but recommended for parallel execution. In addition, the code provides a Python wrapper that allows it to be run and controlled directly from Python.

To verify that the code runs correctly:

```
$ cd tests
$ ./wlcfc
```

This will execute the code using default parameters. A directory named *Output* will be created inside *tests*, where all results are stored, including:

- output data files (e.g., *zetam**, *Bmells**, *Bnk**)
- a log file inside *Output/tmp*

You can consult man page:

```
$ man ../docs/wlcf.1
```

or open with a browser the html file: docs/man/wlcf.html

1.4 License

wlcf is written by A. Aviles et al., and is distributed under the [MIT license](#). If you use this program in research work that results in publications, please cite the following paper:

Abraham Arvizu et al., [arXiv:2048.16847](#)

1.5 Acknowledgements

wlcf use/is based on the following codes or projects:

- [FFTLog](#)
- [The BiHaloFit model of Takahashi](#)
- [Zeno](#)
- [Numerical recipies](#)
- [GSL](#)
- [CLASS](#)

WLCF INSTALLATION

The `wlcf` code is a C-based implementation for computing higher-order correlation functions in cosmology, with optional support for parallel execution via OpenMP and a Python interface for flexible workflows. This section guides you through installing the code, compiling it, and running a first test example.

2.1 Requirements

The code requires:

- A C compiler (recommended: `gcc >= 10`)
- Optional libraries:
 - **GSL** (GNU Scientific Library)
 - **FFTW3**
- Python (optional, for the wrapper interface):
 - `numpy`
 - `cython`

2.2 Compilation

Clone the repository:

```
git clone https://github.com/your-repo/wlcf.git
cd wlcf
```

Edit the file `Makefile_machine` if needed to specify library paths (GSL, FFTW).

Compile the code:

```
make clean
make
```

If GSL is not available, it can be disabled in `Makefile_settings` by setting:

```
USEGSL = 0
```

If compilation is successful, the executable `wlcf` will be created in the main directory.

To verify the installation:

```
cd tests
./wlcfc
```

This runs the code using default parameters.

After execution, the following will be generated:

- *Output/* directory
- Correlation function files:
 - *zetam**
 - *Bmells**
 - *Bnk**
- Log files (inside *Output/tmp*)
- Parameter file: *parameters_null-cballs-usedvalues**

This parameter file can be used as a template for custom runs.

Python wrapper

The code includes a Python interface for easier control and integration.

Build the wrapper

```
make all
```

Install dependencies if needed:

You can use the wrapper as

```
from wlcfcy import wlcfc
w = wlcfc()
w.set(numberThreads=4, verbose=1)
w.Run()
```

Parallel execution (OpenMP)

The code supports shared-memory parallelism using OpenMP.

To control the number of threads:

```
export OMP_NUM_THREADS=8
```

The code may also set the number of threads internally using the parameter *numberThreads*.

HPC example (Perlmutter)

On **NERSC Perlmutter**, a typical workflow in an interactive node is:

```
module load gcc
salloc -N 1 -C cpu -q interactive -t 01:00:00
export OMP_NUM_THREADS=32
cd wlcfc/tests
../wlcfc
```

Notes

- If compilation fails, verify library paths in *Makefile_machine*
- OpenMP requires a compatible compiler (e.g., *gcc* with *-fopenmp*)
- Performance may saturate at high thread counts depending on the workload
- For large runs, always use compute nodes rather than login nodes

PARAMETERS

This section describes the various parameters **wlcf** needs for controlling the computation of the three-point correlation function (3PCF) for weak lensing convergence.

3.1 Parameters related to the cosmological model

- z**
(float) Redshift at which the correlation functions are evaluated.
- h**
(float) Dimensionless Hubble parameter.
- sigma8**
(float) Normalization of the matter power spectrum.
- Omb**
(float) Baryon density parameter.
- Omc**
(float) Cold dark matter density parameter.
- Omnu**
(float, default=0.0) Massive neutrino density parameter.
- ns**
(float) Spectral index of the primordial power spectrum.
- w**
(float, default=-1.0) Dark energy equation of state parameter.

3.2 Parameters to control the input power spectrum

- fnamePS**
(str) [alias: ps] File containing the linear matter power spectrum ($P(k)$).

Use it as:

```
fnamePS=./input/linear_pk.txt
```

3.3 Parameters related to the weak lensing kernel

- Wg**
(int, default=0) Controls how the lensing kernel is defined.

- 0 = Dirac delta at the redshift bin
- 1 = Read kernel from file

fWgchi

(str) [alias: fwgchi] File containing the weak lensing kernel ($W_g(\chi)$).

Use it as:

```
fWgchi=./input/Wg_Takahashi_z05078.txt
```

3.4 Parameters controlling the model and numerical setup

tree_level

(int, default=3) [alias: tlev] Model used for the bispectrum calculation.

- 1 = Standard Perturbation Theory (SPT)
- 2 = Tree-level approximation
- 3 = Effective Field Theory (EFT)
- 4 = Halo Model (HM)

zbin

(float) Redshift bin used in the projection.

mMax

(int) Maximum multipole order for the expansion.

chiQuadSteps

(int) [alias: chiqst] Number of integration steps over comoving distance.

GLpoints

(int) [alias: gl] Number of Gauss–Legendre points.

Nell

(int) Number of angular multipoles.

ellmin

(float) Minimum multipole value.

ellmax

(float) Maximum multipole value.

3.5 Parameters to control output files

rootDir

(str, default="Output") [alias: root] Directory where output files will be written.

prefix

(str, default="run1_") [alias: pre] Prefix for output file names.

path_Bells

(str, default="Bell_outputs") [alias: bellout] Directory where angular correlation functions are stored.

writevectors

(bool, default=true) Whether to write intermediate vectors.

3.6 Miscellaneous parameters

verbose

(int, default=2) [alias: verb] Controls the amount of information printed to the terminal.

- 0 = no output unless there is an error
- 1 = warnings
- 2 = progress information
- 3 = debugging output

verbose_log

(int, default=1) [alias: verblog] Controls logging to file inside *Output/tmp*.

chatty

(int, default=2) Additional verbosity level for internal messages.

numberThreads

(int, default=4) [alias: nthreads] Number of OpenMP threads used during execution.

It is required to enable OpenMP by setting:

```
OPENMPMACHINE = 1
```

in *Makefile_settings* and recompiling the code.

options

(str, default="") [alias: opt] Additional options controlling code behavior.

- It is not necessary to specify all parameters. Only the relevant ones need to be provided. The rest will take default values.
- A parameter file is automatically generated after each run:

```
Output/parameters_null-cballs-usedvalues
```

This file can be used as a template for custom configurations.

- When specifying *rootDir*, avoid using *./* at the beginning or */* at the end if it is a local directory.

LINEAR POWER SPECTRUM FILES

This section describes the format of the linear power spectrum required to compute correlation functions using **wlcf**.

Standard formats

columns-ascii

The linear power spectrum is provided as an ASCII file with two columns:

- k = wavenumber in units of (h, Mpc^{-1})
- $P(k)$ = linear matter power spectrum in units of $(\mathrm{Mpc}/h)^3$

Each row corresponds to a value of (k) and its associated power spectrum.

Example:

```
# k [h/Mpc]    P(k) [(Mpc/h)^3]
1.000000e-03   2.345678e+04
2.000000e-03   1.987654e+04
5.000000e-03   1.234567e+04
...
```

Usage in wlcf

To use a power spectrum file, specify:

```
fnamePS=./input/linear_pk.txt
```

The linear power spectrum can be generated using external Boltzmann solvers such as:

- **CAMB**
- **CLASS**

These tools allow you to compute $(P(k))$ for a given cosmological model.

Make sure to export the power spectrum at the desired redshift and in the correct units.

PRE/POST PROCESSING

This section describes the various pre/pos processing for controlling what the *wlcf* before/after main processing:

5.1 Pre-processing

Pre-processing involves preparing all required inputs before running **wlcf**.

Typical tasks include:

- Generating the **linear power spectrum** ($P(k)$) using external tools (e.g., CAMB or CLASS)
- Preparing the **weak lensing kernel** file (if $Wg=I$)
- Setting cosmological parameters consistently with the input data
- Organizing input files into accessible directories

The accuracy of the final correlation functions strongly depends on the quality and consistency of the input power spectrum and cosmological parameters.

5.2 Post-processing

Post-processing refers to analyzing and visualizing the outputs produced by **wlcf**.

Typical tasks include:

- Reading output files such as:
 - *zetam** (angular correlation functions)
 - *Bmells** (projected bispectrum)
 - *Bnk** (bispectrum in Fourier space)
- Plotting results (e.g., multipoles, angular correlations)
- Comparing results across different cosmologies
- Converting outputs into formats suitable for further analysis

Notes

- Pre-processing is external to **wlcf**; the code assumes inputs are already prepared.
- Post-processing is flexible and depends on the scientific goals.
- For large parameter scans, combining the Python interface with HPC resources is recommended.

ADDON'S

This section describes how to extend the functionality of **wlcf** by adding new features, models, or utilities to the codebase.

The modular structure of the code allows users to include additional functionality through custom header and source files.

Include files

Header files define the interface of new functionalities and must be placed in the appropriate include directories.

To add new features:

1. Create a header file (e.g., *my_module.h*)
2. Place it in one of the include directories:

```
include/  
addons/addons_include/
```

3. Declare functions, structures, or parameters inside the header.

Example

c:

```
#ifndef MY_MODULE_H  
#define MY_MODULE_H  
void my_new_function(double *input, double *output);  
#endif
```

4. Include the header in the relevant source files:

c:

```
#include "my_module.h"
```

Source files

Source files implement the functionality declared in the header files.

To add a new module:

1. Create a source file (e.g., *my_module.c*)
2. Place it in one of the source directories:

```
source/  
addons/
```

3. Implement the functions:

c:

```
#include "my_module.h"
void my_new_function(double *input, double *output) {
// Example computation
*output = input[0] * 2.0;}
```

Compilation

To ensure the new files are compiled:

- Add the source file to the build system (if not automatically included)
- Modify *Makefile* or *Makefile_machine* if necessary

Example (if manual inclusion is needed):

make:

```
SRC += source/my_module.c
```

Integration

To integrate the new functionality into **wlcf**:

- Call your new functions from existing modules (e.g., in *procedures.c*)
- Ensure parameters are passed correctly
- Update relevant headers if new parameters are introduced

PYTHON INTERFACE

This section describes how to interact with **wlcf** using Python, either through the command line interface or via the Cython wrapper.

Using command line interface

The simplest way to run **wlcf** from Python is by calling the executable through the system shell.

Example

python:

```
import os

os.system("../wlcf z=0.5 sigma8=0.8 fnamePS=./input/linear_pk.txt")
```

Alternatively, using *subprocess* :

python:

```
import subprocess

subprocess.run([
    "../wlcf",
    "z=0.5",
    "sigma8=0.8",
    "fnamePS=./input/linear_pk.txt"])
```

This approach allows you to:

- run parameter scans
- automate workflows
- integrate wlcf into larger pipelines

When using the command line interface, parameters must be passed as *key=value* without spaces.

Cython interface

The code provides a **Cython wrapper** that allows direct interaction with wlcf from Python without invoking the executable.

example

python:

```
from wlcipy import wlcif

w = wlcif()

w.set(
    numberThreads=4,
    verbose=1,
    fnamePS="./input/linear_pk.txt")

w.Run()
```

Notes

- The working directory should contain the required input files (e.g., power spectrum).
- Output files are written in the same way as the C executable.
- Some parameters (e.g., number of threads) may be controlled both via Python and internally by the code.

HANDS ON

This section provides a practical example of how to use **wlcf** from a Python notebook. The example follows a complete workflow:

1. Generate a linear matter power spectrum using CAMB.
2. Run **wlcf** through the Python wrapper.
3. Plot the resulting 3-point correlation function multipoles.
4. Compare different theoretical models.

The notebook associated with this tutorial can be used as a starting point for interactive runs and parameter exploration.

8.1 Setting up the working directory

First, import the required Python packages and move to the `tests` directory of the repository:

```
import os
import numpy as np
import matplotlib.pyplot as plt

import camb
from camb import model
from wlcfsy import wlcf

WLCF_TESTS_DIR = "/path/to/wlcf/tests"
os.chdir(WLCF_TESTS_DIR)

os.makedirs("input", exist_ok=True)
os.makedirs("Output", exist_ok=True)
os.makedirs("Bell_outputs", exist_ok=True)
```

8.2 Generating the linear power spectrum

The linear matter power spectrum can be generated with CAMB. In this example we use a T17-like cosmology at redshift $z = 0.5078$:

```
def make_camb_linear_pk(
    output_file="./input/linear_pk_camb_T17_z05078.txt",
    z_pk=0.5078,
    h=0.7,
```

(continues on next page)

(continued from previous page)

```

Omb=0.046,
Omc=0.233,
Omnu=0.0,
ns=0.97,
sigma8=0.82,
w0=-1.0,
kmin=1e-4,
kmax=50.0,
npoints=1000,
):

os.makedirs(os.path.dirname(output_file), exist_ok=True)

H0 = 100.0 * h
ombh2 = Omb * h**2
omch2 = Omc * h**2

pars = camb.CAMBparams()

# For now we assume massless neutrinos, consistent with Omnu = 0.
pars.set_cosmology(
    H0=H0,
    ombh2=ombh2,
    omch2=omch2,
)

pars.InitPower.set_params(
    ns=ns,
    As=2.1e-9
)

pars.set_dark_energy(w=w0)

# Linear power spectrum at the same redshift used by wlcfs/zbin
pars.set_matter_power(
    redshifts=[z_pk],
    kmax=kmax
)

pars.NonLinear = model.NonLinear_none

# First run to compute sigma8 for the trial As
results = camb.get_results(pars)
sigma8_current = results.get_sigma8()[0]

# Rescale As so that CAMB matches the requested sigma8
As_rescaled = 2.1e-9 * (sigma8 / sigma8_current) ** 2

pars.InitPower.set_params(
    ns=ns,
    As=As_rescaled
)

```

(continues on next page)

(continued from previous page)

```

results = camb.get_results(pars)

kh, redshifts, pk = results.get_matter_power_spectrum(
    minkh=kmin,
    maxkh=kmax,
    npoints=npoints,
)

pk_z = pk[0]

np.savetxt(
    output_file,
    np.column_stack([kh, pk_z]),
    fmt="%.10e"
)

print(f"Saved linear P(k) at z={z_pk} to:")
print(output_file)
print(f"sigma8 target = {sigma8}")
print(f"sigma8 CAMB = {results.get_sigma8()[0]:.6f}")
print(f"As rescaled = {As_rescaled:.6e}")

return output_file, kh, pk_z

```

The resulting file is passed to wlcfc through the parameter fnamePS.

8.3 Running wlcfc from Python

The code can be executed directly from Python using the wlcfcpy wrapper:

```

w = wlcfc()
w.set(
    numberThreads=8,
    verbose=2,
    verbose_log=1,

    z=0.5078,
    h=0.7,
    sigma8=0.82,
    Omb=0.046,
    Omc=0.233,
    Omnu=0.0,
    ns=0.97,
    w=-1.0,

    fnamePS="./input/linear_pk_camb_T17_z05078.txt",
    Wg=0,
    fWgchi="./input/Wg_Takahashi_z05078.txt",

    rootDir="Output",
    path_Bells="Bell_outputs",

```

(continues on next page)

(continued from previous page)

```

prefix="halo_model_z05078_",

tree_level=4,
zbin=0.5078,
mMax=5,
chiQuadSteps=300,
GLpoints=64,
Nell=128,
ellmax=10000,
ellmin=0.001,
writevectors=0,
chatty=2,
options=""
)

w.Run()

```

Here `tree_level=4` selects the Takahashi/Halo-model branch.

8.4 Plotting the 3PCF multipoles

The output files `zetam*.txt` contain the multipoles $\zeta_m(\theta_1, \theta_2)$. These can be visualized as 2D maps:

```

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import LogNorm

path = "/home/sadi/LSST/test/wlcf-main/tests/Bell_outputs"
prefix = "halo_model_z05078_"

theta = np.loadtxt(f"{path}/{prefix}theta_array.txt") * 180 / np.pi * 60
Theta2, Theta1 = np.meshgrid(theta, theta)

fig = plt.figure(figsize=(19, 4))

gs = fig.add_gridspec(
    nrows=1,
    ncols=6,
    width_ratios=[1, 1, 1, 1, 1, 0.06],
    wspace=0.25
)

norm = LogNorm(vmin=1e-11, vmax=1e-8)

for m in range(5):
    ax = fig.add_subplot(gs[0, m])

    zeta = np.abs(np.loadtxt(f"{path}/{prefix}zetam{m}.txt"))

    im = ax.pcolormesh(
        Theta2,

```

(continues on next page)

(continued from previous page)

```

        Theta1,
        zeta,
        shading="auto",
        cmap="RdYlBu_r",
        norm=norm
    )

    ax.set_xscale("log")
    ax.set_yscale("log")
    ax.set_xlim(10, 200)
    ax.set_ylim(10, 200)
    ax.set_aspect("equal", adjustable="box")
    ax.set_title(fr"$m={m}$", fontsize=14)

    if m == 0:
        ax.set_ylabel(r"$\theta_1$ [arcmin]", fontsize=13)

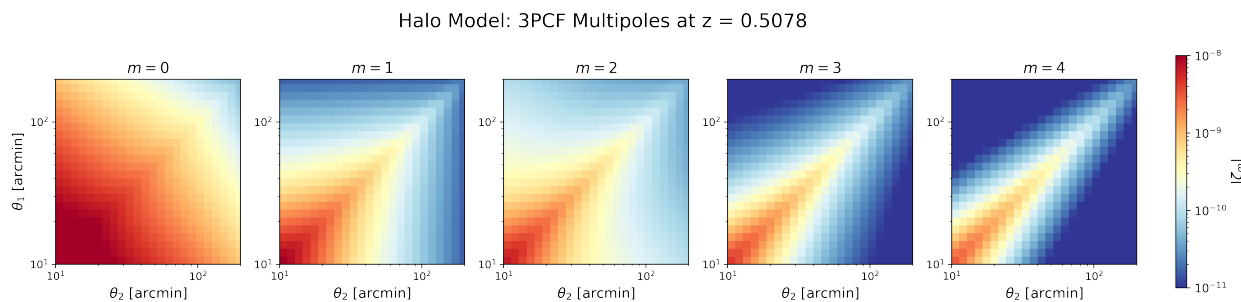
    ax.set_xlabel(r"$\theta_2$ [arcmin]", fontsize=13)

# Colorbar outside the plots
cax = fig.add_subplot(gs[0, -1])
cbar = fig.colorbar(im, cax=cax)
cbar.set_label(r"$|\zeta_m|$", fontsize=13)
cbar.ax.tick_params(labelsize=11)

fig.suptitle(
    "Halo Model: 3PCF Multipoles at  $z = 0.5078$ ",
    fontsize=18,
    y=1.02
)

plt.savefig("halo_model_3pcf.png", dpi=300, bbox_inches="tight")
plt.show()

```



Or we can plot

```

path = "/home/sadi/LSST/test/wlcf-main/tests/Bell_outputs"
prefix = "halo_model_z05078_"

theta = np.loadtxt(fr"{path}/{prefix}theta_array.txt") * 180/np.pi * 60

theta2_targets = [13, 72, 169]
m_values = [0, 1, 2]

```

(continues on next page)

(continued from previous page)

```

fig, axes = plt.subplots(
    len(theta2_targets),
    len(m_values),
    figsize=(9, 9),
    sharex=True
)

for i, theta2_fixed in enumerate(theta2_targets):
    idx = np.argmin(np.abs(theta - theta2_fixed))
    theta2_actual = theta[idx]

    for j, m in enumerate(m_values):
        ax = axes[i, j]

        zeta = np.abs(np.loadtxt(f"{path}/{prefix}zetam{m}.txt"))

        # theta2 fixed, theta1 varies
        y = zeta[:, idx]

        ax.plot(theta, y, color="black", marker="o", markersize=3, lw=1)

        ax.set_xscale("log")
        ax.set_yscale("log")

        ax.set_xlim(10, 200)

        if i == 0:
            ax.set_title(fr"$m={m}$")

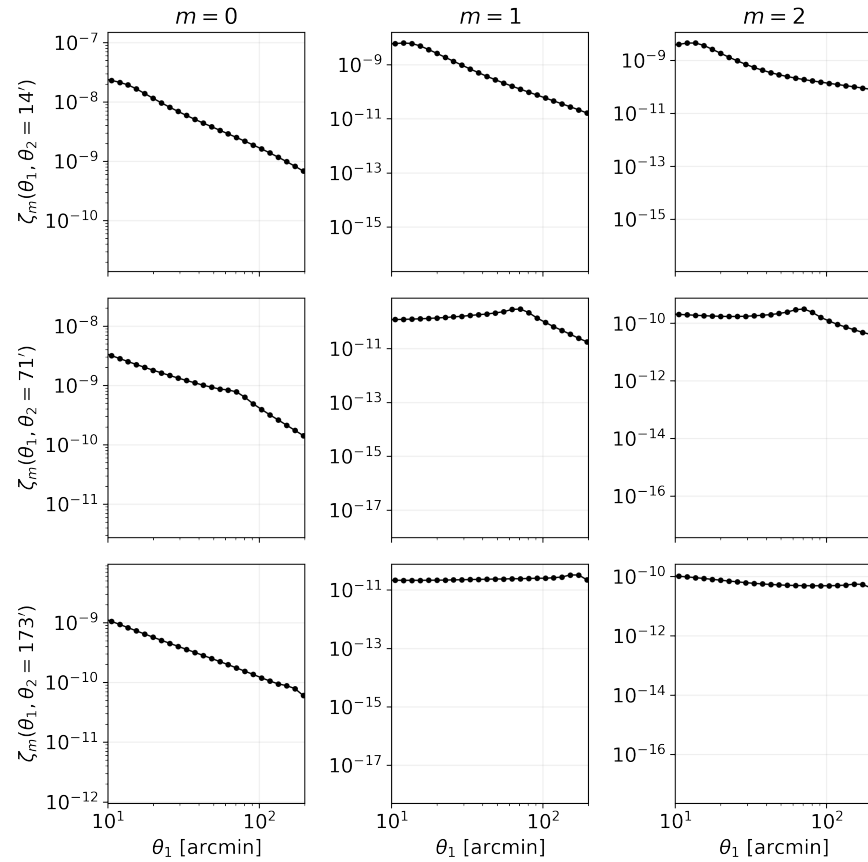
        if j == 0:
            ax.set_ylabel(
                fr"$\zeta_m(\theta_1, \theta_2={theta2_actual:.0f})$"
            )

        if i == len(theta2_targets) - 1:
            ax.set_xlabel(r"$\theta_1$ [arcmin]")

        ax.grid(alpha=0.2)

plt.tight_layout()
plt.savefig("halo_model_slice_3pcf.png", dpi=300, bbox_inches="tight")
plt.show()

```



8.5 Comparing different models

We can run several theoretical models by changing `tree_level`:

```
base_params = dict(
    numberThreads=8,
    verbose=2,
    verbose_log=1,

    # T17 / paper cosmology
    z=0.5078,
    h=0.7,
    sigma8=0.82,
    Omb=0.046,
    Omc=0.233,
    Omnu=0.0,
    ns=0.97,
    w=-1.0,

    # Make sure this P(k) was generated with the same cosmology
    fnamePS="./input/linear_pk_camb_T17_z05078.txt",

    # Redshift bin
    Wg=0,
```

(continues on next page)

(continued from previous page)

```

fWgchi="./input/Wg_Takahashi_z05078.txt",

rootDir="Output",
path_Bells="Bell_outputs",

zbin=0.5078,
mMax=5,
chiQuadSteps=300,
GLpoints=64,
Nell=128,
ellmax=10000,
ellmin=0.001,
writevectors=0,
chatty=2,
options=""
)

models = {
    1: "SPT",
    2: "Tree",
    3: "EFT",
    4: "Halo_model"
}

for tree_level, model_name in models.items():

    print(f"Running {model_name} model with tree_level={tree_level}")

    params = base_params.copy()
    params["tree_level"] = tree_level
    params["prefix"] = f"{model_name.lower()}_z05078_"

    w = wlcfc()
    w.set(params)
    w.Run()

    print(f"Finished {model_name}")

```

We plot the different cases

```

plt.rcParams.update({
    "font.size": 14,
    "axes.titlesize": 16,
    "axes.labelsize": 14
})

path = "/home/sadi/LSST/test/wlcfc-main/tests/Bell_outputs"

models = {
    "SPT": "spt_z05078_",
    "Tree": "tree_z05078_",
    "EFT": "eft_z05078_",

```

(continues on next page)

(continued from previous page)

```

    "Halo Model": "halo_model_z05078_",
}

m_values = range(5)

theta = np.loadtxt(f"{path}/spt_z05_theta_array.txt") * 180 / np.pi * 60
Theta2, Theta1 = np.meshgrid(theta, theta)

nrows = len(models)
ncols = len(m_values)

fig = plt.figure(figsize=(20, 13))

gs = fig.add_gridspec(
    nrows=nrows,
    ncols=ncols + 1,
    width_ratios=[1, 1, 1, 1, 1, 0.05],
    wspace=0.2,
    hspace=0.25
)

norm = LogNorm(vmin=1e-11, vmax=1e-8)

for i, (model_name, prefix) in enumerate(models.items()):
    for j, m in enumerate(m_values):

        ax = fig.add_subplot(gs[i, j])

        zeta = np.abs(np.loadtxt(f"{path}/{prefix}zetam{m}.txt"))

        im = ax.pcolormesh(
            Theta2,
            Theta1,
            zeta,
            shading="auto",
            cmap="RdYlBu_r",
            norm=norm
        )

        ax.set_xscale("log")
        ax.set_yscale("log")
        ax.set_xlim(10, 200)
        ax.set_ylim(10, 200)

        if i == 0:
            ax.set_title(fr"$m={m}$")

        if j == 0:
            ax.set_ylabel(f"{model_name}\n" + r"$\theta_1$ [arcmin]")

        if i == nrows - 1:
            ax.set_xlabel(r"$\theta_2$ [arcmin]")

```

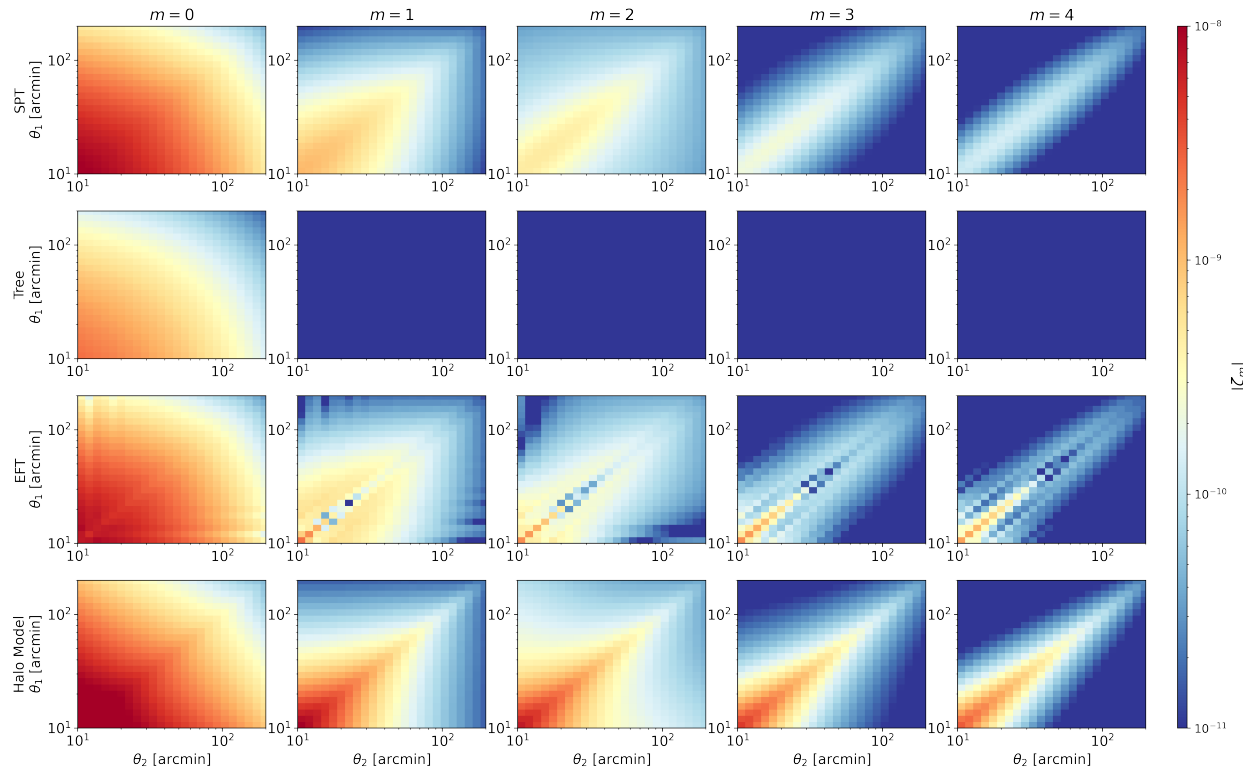
(continues on next page)

(continued from previous page)

```

cax = fig.add_subplot(gs[:, -1])
cbar = fig.colorbar(im, cax=cax)
cbar.set_label(r"$|\zeta_m|$", fontsize=16)
cbar.ax.tick_params(labelsize=12)
plt.savefig("models_3pcf.png", dpi=300, bbox_inches="tight")
plt.show()

```



3-POINT CORRELATION FUNCTIONS

wlcf computes **three-point correlation functions (3PCF)** for scalar fields, such as the weak lensing convergence κ . These higher-order statistics provide access to non-Gaussian information in the large-scale structure of the Universe.

In particular, the code evaluates the correlation:

$$\langle \kappa(\hat{n}_1), \kappa(\hat{n}_2), \kappa(\hat{n}_3) \rangle$$

using a plane-wave expansion and a harmonic decomposition.

Harmonic base

The angular dependence of the 3PCF is expressed in a **harmonic basis**, where the signal is decomposed into multipoles. This allows for an efficient and structured representation of the correlation function.

The expansion is performed in terms of angular modes labeled by an index m , leading to a set of coefficients:

$$\zeta_m(\theta_1, \theta_2)$$

where:

- θ_1, θ_2 are angular separations
- $m = 0, 1, \dots, m_{\max}$ is the multipole index

Output

The results of the harmonic decomposition are stored in files:

```
zetam0.txt
zetam1.txt
...
zetamN.txt
```

along with the corresponding angular grid:

```
theta_array.txt
```

These outputs can be used to reconstruct or visualize the full three-point correlation function.

INDICES AND TABLES

- `genindex`
- `modindex`
- `search`